Title. Compound-Truck Cross-Docking Scheduling with Arrival Time Windows: A Bottleneck-Guided Iterated Local Search with Exact-Solver Verification

Abstract

This paper considers a multi-door cross-dock in which a truck can be partially unloaded and then leave as an outbound carrier. Existing compound-truck models place every truck at the dock at time zero and optimize makespan. Here, trucks have individual release times and soft departure due dates, and the objective also charges tardiness. We formulate the problem as a mixed-integer program and as a CP-SAT model, and use separate train, tuning, and test seeds in the computational study. Our method, VAA-GILS, starts from a Vogel-style construction and uses best-improvement descent throughout; its additional moves focus on the door that determines makespan or on the most tardy truck, with simulated-annealing acceptance and kick restarts for diversification. On the subsets for which CP-SAT produced a reference, the resulting objectives were 0.1–0.6% from the CP-SAT incumbents, and the small no-window references were proven optimal. A GILS run took 0.1–2.3 seconds, versus 76–376 seconds for CP-SAT. For the larger time-window cases, CP-SAT found no feasible schedule within 600 seconds, while GILS returned the best solutions among the methods tested. Ablation is less favorable to learning: descent, the bottleneck moves, and kick restarts account for the improvement, whereas changing the initial solution, retaining stochastic acceptance, or replacing uniform operator sampling with tabular Q-learning or a transfer DQN changed the objective by no more than 0.2 percentage points, and the learned selectors did not win at any tested budget.

Keywords: cross-docking; truck scheduling; time windows; iterated local search; constraint programming; guided local search

1. Introduction

Cross-docking centers transfer products from inbound to outbound trucks with little or no storage, reducing inventory and lead time (Van Belle et al., 2012). In the compound-truck variant introduced by Joo and Kim (2013) and refined by Shahmardan and Sajadieh (2020), a truck may serve both roles: a compound truck arrives loaded with products for several destinations, is *partially* unloaded — retaining the demand of one destination — and then reloads that destination's remaining demand before departing as an outbound truck. Partial unloading was shown to reduce makespan by up to 56% relative to full unloading (Shahmardan & Sajadieh, 2020), making this model practically attractive for less-than-truckload consolidation networks.

The simultaneous-arrival assumption is restrictive for scheduled docks, where release times differ by truck. A makespan-only objective is also incomplete when outbound departures have due dates. Release times, time windows, and due-date objectives have been studied for conventional inbound/outbound trucks (Assadi & Bagheri, 2016; Bodnar et al., 2017; Konur & Golias, 2013a, 2013b;

Ladier & Alpan, 2016b; Larbi et al., 2011; Molavi et al., 2018; Van Belle et al., 2013; Xi et al., 2020); our contribution is to integrate known release times and soft due dates into the partial-unloading compound-truck setting. To our knowledge, this combination has not previously been formulated. Section 3 gives the big-M model, its reified CP-SAT counterpart, and lower bounds. The instance generator and the train/tuning/test seed split are included so that the computational protocol can be reproduced.

For this problem we develop VAA-GILS. The method keeps the VAA construction and generic moves used in earlier work (Shahmardan & Sajadieh, 2020), but adds a full best-improvement descent, bottleneck-specific moves, and restarts. Its default selector is uniform rather than learned. The exact-solver comparison is deliberately limited to the subsets on which CP-SAT returned an incumbent; only the small no-window subset has proven optima. Elsewhere we compare against the best solutions found by the tested methods.

Much of the paper is devoted to identifying which parts of GILS matter. The operator-pool experiment gives a monotone improvement when critical and tardy moves are added ($p < 0.002$). Removing descent causes the largest loss, followed by removing kick restarts. In contrast, the final quality changes little when VAA is replaced by a random feasible start or when simulated-annealing acceptance is removed. We obtained a similar result for operator selection: neither tabular Q-learning nor a transfer-trained DQN gave a meaningful gain over uniform sampling over budgets of 50–3,000 iterations.

These comparisons concern our reimplementation and benchmark. They should not be read as a replication claim about Shahmardan and Sajadieh (2020), whose experimental setting and code differ. Their narrower implication is methodological: when a learned selector is embedded in a strong search engine, a uniform, equal-budget control is needed to establish its incremental value.

2. Related work

Cross-dock truck scheduling with temporal constraints. Boysen and Fliedner (2010) classify dock-scheduling problems; Van Belle et al. (2012) and Ladier and Alpan (2016a) survey the field and its industry gap. Several deterministic models already incorporate temporal constraints for conventional inbound and outbound trucks. Van Belle et al. (2013) consider multiple docks, time windows, and tardiness; Assadi and Bagheri (2016) use truck ready times and minimize outbound earliness and tardiness; and Bodnar et al. (2017) combine time windows with mixed-service doors. Molavi et al. (2018) instead impose hard outbound due dates, while Rijal et al. (2019) integrate truck scheduling and door assignment and include tardiness in a mixed-service-door setting. These studies establish that releases and due-date performance are not new to cross-dock scheduling. They retain the conventional separation between inbound and outbound trucks, however, and do not model a partially unloaded truck that subsequently becomes a carrier.

Incomplete and uncertain arrival information. Arrival uncertainty is a separate, well-developed stream. Konur and Golias study both bounded arrival windows (2013a) and cost-stable schedules under unknown arrivals (2013b). Larbi et al. (2011) distinguish full, partial, and absent information on inbound arrivals; Ladier and Alpan (2016b) develop robust schedules with time windows; and Xi et al. (2020) address uncertain arrival and operation times through two-stage conflict robust optimization. Our model is deterministic: each release time is known when the schedule is constructed. These uncertainty models therefore provide natural extensions rather than direct substitutes for the problem studied here.

Compound trucks and partial unloading. Joo and Kim (2013) introduced compound trucks with exclusive door service. Shahmardan and Sajadieh (2020) added partial unloading, a mixed door service mode, a MILP, a VAA-based constructive heuristic, and simulated annealing variants whose neighborhood selection is driven by multi-armed bandits or Q-learning; their SA-RL5 variant is our main baseline. Follow-up work on this model line is sparse, and none considers arrival times or due dates.

Closest neighboring problem. Li et al. (2026) study integrated truck assignment and scheduling with *mixed service mode docks* and time windows, solved by a Q-learning-based adaptive large neighborhood search. Their flexibility lives at the dock level (a door may serve either direction), while trucks remain purely inbound or outbound and transfers are routed through storage by AGVs; in our problem the flexibility lives at the truck level (compound trucks, direct door-to-door transfer). Their Q-learning, like the original model's, is trained online within each instance.

Learned metaheuristics. Neural construction policies (Kool et al., 2019; Zhang et al., 2020) and learned operator selection (via bandits, tabular Q (Watkins & Dayan, 1992), or deep Q-networks (Mnih et al., 2015)) have been reported to improve many metaheuristics. The experiments below examine a less commonly reported outcome: after strengthening the underlying local search, the learned selector adds little on this benchmark.

3. Problem definition

3.1 Base model

Let I be the set of compound trucks, F the outbound trucks, D the destinations ($|I| + |F| = |D|$), K the product types, and M the dock doors ($|I| \leq |M|$). A compound truck i carries f_{idk} units of product k for destination d; the unit handling time is t_k, and moving one batch between doors m and n takes t_{mn}. Truck i needs changeover times DE_i (docking) and DL_i (undocking). Define the handling time $h_{id} = \sum_k f_{idk} t_k$.

Decisions: (i) each compound truck is assigned one *retained* destination and one door (at most one compound truck per door); (ii) each outbound truck is

assigned one destination and one door; (iii) outbound trucks sharing a door are sequenced. Each destination is served by exactly one truck (its *carrier*). A compound truck unloads everything except its retained destination's demand, transfers move products to carrier doors, and each carrier loads its destination's demand collected from all compound trucks.

Timing follows the evaluator semantics of Shahmardan and Sajadieh (2020): a compound truck's unload finishes at $r_i + DE_i + \sum_{d' \neq d} h_{id'}$; a destination is ready at its carrier door when the last contributing transfer arrives; loading starts at the maximum of own unload completion and destination readiness for compound carriers, and at the maximum of door availability, destination readiness, and r_f for outbound carriers. The undocking time DL is added after loading. The makespan τ is the largest completion time.

3.2 Mixed-integer formulation

We state the formulation implemented in our MILP. Binary x_{idm} equals one when compound truck $i \in I$ retains destination $d \in D$ at door $m \in M$; z_{fdm} analogously assigns outbound truck $f \in F$. Define $X_{im} = \sum_d x_{idm}$, $Z_{fm} = \sum_d z_{fdm}$, and $Z_{fd} = \sum_m z_{fdm}$. Binary b_{fgm} equals one when outbound f precedes g on door m. Continuous U_i, C_i, S_f, C_f are compound unload completion, compound completion, outbound start, and outbound completion; $C_{\max}$ is makespan. Let $H_i = \sum_d h_{id}$, $L_{id} = \sum_{j \neq i} h_{jd}$, $L_d = \sum_i h_{id}$, and let B be a valid upper bound on the scheduling horizon.

Carrier and door assignments satisfy

$$\sum_{d,m} x_{idm} = 1 \quad \forall i, \qquad \sum_{d,m} z_{fdm} = 1 \quad \forall f, \tag{2}$$

$$\sum_{i,m} x_{idm} + \sum_{f,m} z_{fdm} = 1 \quad \forall d, \tag{3}$$

$$\sum_{i,d} x_{idm} \leq 1 \quad \forall m. \tag{4}$$

Compound timing and incoming-transfer precedence are

$$U_i = r_i + DE_i + H_i - \sum_{d,m} h_{id} x_{idm} \quad \forall i, \tag{5}$$

$$C_i \geq U_i + L_{id} + DL_i - B(1 - x_{idm}) \quad \forall i, d, m, \tag{6}$$

$$C_i \geq U_j + t_{nm} + L_{id} + DL_i - B(2 - x_{idm} - X_{jn}) \tag{7}$$

for every i, d, m, n and contributing $j \neq i$ with $\sum_k f_{jdk} > 0$. Thus a compound carrier waits for every source transfer. For outbound carriers,

$$S_f \geq r_f \quad \forall f, \tag{8}$$

$$C_f \geq S_f + DE_f + L_d + DL_f - B(1 - Z_{fd}) \quad \forall f, d, \tag{9}$$

$$S_f \geq U_i + t_{nm} - B(2 - z_{fdm} - X_{in}) \tag{10}$$

for every f, d, m, n and contributing i with $\sum_k f_{idk} > 0$. An outbound truck also waits for a compound truck assigned to the same door:

$$S_f \geq C_i - B(2 - Z_{fm} - X_{im}) \quad \forall f, i, m. \tag{11}$$

For each outbound pair $f < g$ sharing door m,

$$b_{fgm} + b_{gfm} \geq Z_{fm} + Z_{gm} - 1, \tag{12}$$

with $b_{fgm} \leq Z_{fm}$ and $b_{fgm} \leq Z_{gm}$. The associated non-overlap constraints are

$$S_g \geq C_f - B(1 - b_{fgm}), \qquad S_f \geq C_g - B(1 - b_{gfm}). \tag{13}$$

Finally,

$$C_{\max} \geq C_i \quad \forall i, \qquad C_{\max} \geq C_f \quad \forall f. \tag{14}$$

Equations (2)–(14) model carrier assignment, compound-door exclusivity, transfer precedence, door capacity, and outbound sequencing. The CP-SAT model uses the same semantics but replaces the big-M disjunctions with enforcement literals. Fixing all assignments to a feasible schedule reproduces the independent evaluator's completion times, providing a model-fidelity check.

3.3 Time-window extension and objective

Each truck $q \in I \cup F$ has release time $r_q \geq 0$ and soft due date $\bar{d}_q$. Introduce $T_q \geq 0$ with

$$T_q \geq C_q - \bar{d}_q. \tag{15}$$

The optimization objective is

$$\min\ C_{\max} + \lambda \sum_{q \in I \cup F} T_q. \tag{16}$$

We use $\lambda = 1$; $r_q = 0$ and $\bar{d}_q = \infty$ recover the base model. CP-SAT scales time exactly to 0.01 units and returns both an incumbent and a proven lower bound. We call an incumbent optimal only when the solver proves equality with its bound.

3.4 Combinatorial lower bounds

For gap reporting when exact bounds are weak we use two valid bounds whose maximum bounds τ: (i) a *critical-chain* bound — for each truck, the fastest possible completion over all retained-destination choices, relaxing all door contention; (ii) a *door-area* bound — the total unavoidable door occupancy divided by $|M|$. A structural tardiness bound follows by applying (i) per truck against its due date; since both terms bound their objective components for any feasible solution, $LB_\tau + \lambda LB_T$ is a valid bound on (16).

4. VAA-GILS: guided iterated local search

VAA-GILS instantiates the iterated local search framework (Lourenço et al., 2019) and is deterministic apart from the sampling inside operators and acceptance. One run proceeds as follows.

Construction. The VAA heuristic of Shahmardan and Sajadieh (2020) assigns compound trucks to retained destinations by Vogel-style regret (Korukoğlu & Ballı, 2011), assigns remaining destinations to outbound trucks by completion-time priority, seeds central doors with the first assigned trucks, and inserts remaining outbound trucks into the door with the earliest finish time. The regret assignment and priority ranking retain the base heuristic's processing-time logic; once doors are assigned, all finish-time and insertion computations enforce the release times.

Descent. A best-improvement local search over three move families: relocation of any outbound truck to any door position, compound door swaps and moves to free doors, and destination swaps between any two trucks. Descent terminates in a local optimum of this neighborhood. It is applied to the initial solution, to every new incumbent, and to the final solution.

Iterated search. For 1,000 iterations: an operator is drawn by the selection policy (Section 4.1); the operator generates one candidate from the current solution; if the candidate improves the global best, descent is applied and the incumbent is updated; acceptance follows simulated annealing (Kirkpatrick et al., 1983) with initial temperature $T_0 = \max\{1, 0.05J(s_0)\}$ and geometric cooling $T \leftarrow 0.995T$.

Restart and reheating. Let s^* be the current best solution and $J(s^*)$ its objective. If 30 consecutive candidate evaluations produce no new best, the current solution is replaced by a copy of s^* and then perturbed by three successive kicks. Each kick independently draws one of the seven generic neighborhood operators with equal probability and applies the sampled move to the result of the preceding kick. No descent is performed at the restart. After the iteration's normal cooling, the temperature is reheated according to

$$T \leftarrow \max\{T,\ 0.02J(s^*)\}. \tag{17}$$

Thus reheating sets a lower temperature floor relative to the best objective; it does not reset the temperature to T_0 and never lowers the current temperature. The counters for iterations since the last best solution and for consecutive worsening candidates are both reset to zero after the restart.

Operator pool. Seven generic neighborhood moves from Shahmardan and Sajadieh (2020) (destination swaps, door swaps, insertions; the base model numbers its moves k1–k8 but defines no k5, so seven are implemented) plus four *guided* operators that first identify the current bottleneck and then perform a best-of mini-search around it: (g1) relocate the last outbound truck on the critical door to its best door/position; (g2) swap the critical truck's destination against all trucks; (g3)/(g4) the analogous moves anchored on the most tardy truck, falling back to (g1)/(g2) when no truck is late. A guided move costs tens of evaluations; a fast incremental evaluator (tens of microseconds per evaluation) makes both descent and guided moves cheap.

4.1 Operator-selection policies

The selection policy is a plug-in, which lets us isolate its contribution. The engine's default is uniform random selection; the two learned policies below are evaluated only as ablation arms (Section 6.2), not as part of the proposed method.

- **Uniform** (default): uniform random choice over the operator pool.
- **Tabular Q** (the policy of SA-RL5 in Shahmardan and Sajadieh, 2020): Q-learning (Watkins & Dayan, 1992) over five stagnation-bin states, trained online within the run; ε-greedy with roulette exploitation, shaped reward (2 for a new incumbent, 1 for a non-worsening move, 0 otherwise).
- **Transfer DQN**: a two-layer deep Q-network (Mnih et al., 2015) over a 27-dimensional scale-invariant state (instance descriptors such as compound-truck fraction, flow concentration, window tightness; search descriptors such as progress, temperature, stagnation, door-load imbalance; and per-operator recent success rates). It is trained offline on 500 runs over a *training* instance pool (sizes S/M, all flow patterns and window levels) and applied zero-shot ($\varepsilon = 0$, no updates) to unseen test instances.

4.2 Relation to the base model of Shahmardan and Sajadieh (2020)

VAA-GILS shares the construction heuristic and the seven generic neighborhoods of the base model (Shahmardan & Sajadieh, 2020), and like it applies exactly one operator per iteration. The differences are structural, and Table 1 makes them explicit: Shahmardan and Sajadieh (2020) is a *pure* simulated-annealing method in which the reinforcement-learning policy chooses which single move to apply and the step ends at the SA accept/reject; VAA-GILS embeds that same one-operator step inside an iterated local search, adding bottleneck-guided operators, a best-improvement descent after every improving move, and kick restarts with reheating.

Table 1. Base model (Shahmardan & Sajadieh, 2020) versus VAA-GILS.

Aspect	Base SA-RL5 (Shahmardan & Sajadieh, 2020)	VAA-GILS (ours)
Overall framework	Pure simulated annealing	Iterated local search (SA only as acceptance)
Operators per iteration	One	One (same)
Operator pool	7 neighborhoods (numbered k1–k8, k5 unused)	Those 7 + 4 bottleneck-guided (g1–g4)
Guided-move mechanics	Roulette-wheel single move (k6–k8)	Deterministic best-of scan around the makespan-critical / most-tardy truck
Which operator fires	Online Q-learning / roulette	Uniform random (learned selection kept only as ablation; no benefit)
After an improving move	Nothing — the SA step is the whole iteration	Full best-improvement descent to a local optimum
Stagnation handling	None	Kick restart from best + reheating
Acceptance	SA (Metropolis)	SA (Metropolis) — shown dispensable by ablation (Section 6.2)
Construction	VAA	VAA, extended with release times
Objective	Makespan	Makespan + $\lambda\cdot$ total tardiness (time windows)

The most consequential difference in Table 1 occurs after an improving move.

In VAA-GILS, that move triggers descent, so the selected operator often begins an improvement that is completed by several further moves. The ablation later shows that this descent accounts for most of the quality gain. Operator choice is a separate issue: Shahmardan and Sajadieh (2020) learn this choice, whereas our uniform control reaches essentially the same final quality as the learned selectors.

5. Experimental design

Instances. We follow the size regime of Shahmardan and Sajadieh (2020): $(|I|, |D|, |M|)$ = S $(6, 9, 6)$, M $(12, 18, 12)$, and L $(20, 30, 20)$, with compound-truck fraction 2/3. Flows f_{idk} are uniform integers in $[0, 20]$ with three product types; unit times follow $t_k \sim U[1, 5]$; doors lie on a random 100×100 layout. Window levels are *none* ($r = 0$, $\bar{d} = \infty$), *medium*, and *tight*: releases follow $r \sim U[0, \rho H]$ and $\bar{d} = r + \delta H$, where H estimates the unconstrained makespan and $(\rho, \delta) = (0.25, 0.60)$ and $(0.50, 0.35)$, respectively.

Protocol. Seeds are split into disjoint train / tuning / test pools. All algorithm design and hyperparameters were frozen on the tuning pool; the DQN was trained on the train pool; every number below is the single test-pool run. The main solution-quality comparison (Table 2 and its Wilcoxon tests) uses 20 instances per cell $\times$ 5 replications (9 cells: 3 sizes $\times$ 3 window levels); the controlled ablations (Tables 3–5 and Figure 3) keep the original 5-instance design, since they are equal-budget, equal-structure decompositions rather than generalization claims. CP-SAT runs once per instance with 8 threads: 300 s on S (5 instances) and 600 s on M and L (2 instances per cell). $\lambda = 1$ on window cells.

Statistics. The independently generated instance is the experimental unit. For stochastic methods, we first average the five replications within each instance and method, then apply two-sided Wilcoxon signed-rank tests to paired instance means. Zero differences are omitted by the signed-rank test, so the reported n can be smaller than the number of paired instances. Replications quantify algorithmic variability but are not treated as independent samples.

6. Results

6.1 Solution quality

Scope of claims. We report proximity to CP-SAT incumbents on the S cells (300 s per instance) and M-none (600 s, two instances). Only the five S-none incumbents were proven optimal, so only that subset supports an optimality-gap statement; all other exact-solver comparisons are incumbent gaps. On the remaining cells CP-SAT returns no feasible solution within 600 s; there our claim is dominance over the tested baselines, and the reported solutions are, to our knowledge, the best known. Accordingly, Table 2 reports the gap to the *best-known* solution per instance (Δ_{bk}, over all methods, all budgets, and CP-SAT) as the primary quality measure, and the gap to the CP-SAT reference where one exists.

Table 2. Test-pool results. Mean obj ± std and Δ_{bk} are over 20 instances × 5 reps per cell (Phase D1); Δ_{bk} is the mean gap to the per-instance best-known solution (over all methods and CP-SAT where available). vs CP-SAT is over the exact-referenced subset only (5 per S cell — S-none: proven optima; 2 for M-none) and is unchanged from the original exact runs.

Cell	Method	Mean obj ± std	Δ_{bk} (%)	vs CP-SAT (%)
S-none	VAA	1497.5 ± 400.3	6.60	+6.49
	Paper-SA-RL5	1423.3 ± 383.6	1.03	+1.21
	GILS-uniform	1411.8 ± 382.7	0.16	+0.21
	GILS-tabular	1412.9 ± 381.3	0.28	+0.33
	GILS-DQN	1416.1 ± 382.4	0.51	+0.75
S-medium	VAA	5447.5 ± 1468.9	3.89	+3.33
	GILS-uniform	5255.7 ± 1413.3	0.08	+0.11
	GILS-tabular	5259.1 ± 1412.3	0.15	+0.18
	GILS-DQN	5270.8 ± 1415.3	0.38	+0.55
S-tight	VAA	9276.8 ± 2387.6	2.90	+2.17
	GILS-uniform	9010.8 ± 2206.1	0.09	+0.21
	GILS-tabular	9011.9 ± 2206.7	0.11	+0.17
	GILS-DQN	9022.0 ± 2205.3	0.23	+0.34
M-none	VAA	3030.9 ± 903.6	3.73	+4.80
	Paper-SA-RL5	2976.4 ± 864.4	1.86	+0.99
	GILS-uniform	2928.9 ± 842.6	0.29	+0.13
	GILS-tabular	2932.1 ± 846.3	0.37	+0.11
	GILS-DQN	2932.5 ± 845.5	0.40	+0.19
M-medium	VAA	23025.5 ± 4075.4	2.56	—
	GILS-uniform	22526.4 ± 3960.5	0.29	—
	GILS-tabular	22515.5 ± 3964.0	0.24	—
	GILS-DQN	22540.3 ± 3956.4	0.36	—
M-tight	VAA	38903.7 ± 11283.5	1.25	—
	GILS-uniform	38528.0 ± 11078.0	0.19	—
	GILS-tabular	38512.1 ± 11059.5	0.16	—
	GILS-DQN	38543.6 ± 11077.6	0.24	—
L-none	VAA	5492.3 ± 1482.2	2.21	—
	Paper-SA-RL5	5479.7 ± 1449.5	1.97	—
	GILS-uniform	5388.4 ± 1427.3	0.27	—
	GILS-tabular	5385.5 ± 1424.6	0.22	—
	GILS-DQN	5386.1 ± 1425.2	0.23	—
L-medium	VAA	55611.1 ± 18087.3	1.52	—
	GILS-uniform	54846.4 ± 17523.5	0.10	—
	GILS-tabular	54833.4 ± 17526.8	0.07	—
	GILS-DQN	54848.8 ± 17522.9	0.11	—
L-tight	VAA	119671.9 ± 32475.3	0.77	—
	GILS-uniform	118901.4 ± 31781.1	0.08	—
	GILS-tabular	118865.6 ± 31788.4	0.05	—

Cell	Method	Mean obj ± std	Δ_{bk} (%)	vs CP-SAT (%)
	GILS-DQN	118888.8 ± 31789.1	0.07	—

On every cell with an exact reference, GILS approaches it (see the final column of Table 2): within 0.21–0.75% of the five proven optima on S-none, within 0.11–0.55% of the 300 s CP-SAT incumbents on S-medium and S-tight, and within 0.11–0.19% of the 600 s incumbents on M-none, which the best GILS runs match or beat. The ordering GILS < Paper-SA-RL5 < VAA holds in every cell where the baseline is defined; over the 20-instance test set, Wilcoxon tests on paired instance means confirm GILS-tabular beats VAA (+2.64% mean, $n = 180$, $p < 10^{-4}$) and Paper-SA-RL5 (+1.33%, $n = 60$, $p < 10^{-4}$). The combinatorial lower bounds are informative on no-window cells but loose elsewhere: the gap of the *best-known solution itself* to the bound is 0.0% on S-none (optimality proven), 18.5–51.5% on the other S cells, 33–34% on M-none and L-none, and 171–273% on the time-window cells at M and L. These figures measure the bound, not the solutions; tightening bounds for large time-window instances is left as future work.

Runtime. Mean per-run times of GILS (1,000 iterations): 0.11 s (S), 0.52 s (M), 2.3 s (L). CP-SAT: 76 s (S, mean incl. early optimality proofs), 237 s (M), 376 s (L). GILS attains its quality at roughly 70–700× less computation.

6.2 Where does the quality come from?

All ablations use 1,000 iterations. We change one part of the engine at a time: the available moves, the search components, or the operator-selection rule. Uniform sampling is retained unless the selector itself is under study. Thus, the paired comparisons differ only in the component named in each table.

Operator pool (guided operators). We fix the selection policy to uniform and vary only the operator pool: *generic* (the seven paper neighborhoods), *critical* (generic plus the two makespan-critical guided operators g1, g2), and *full* (critical plus the two tardiness-guided operators g3, g4). The mean gap to the per-instance best-known solution is monotone — generic ≥ critical ≥ full — in every one of the nine cells (e.g., S-none 0.45 / 0.17 / 0.10; M-medium 0.45 / 0.37 / 0.28). Table 3 gives the paired tests.

Table 3. Operator-pool ablation (two-sided Wilcoxon on paired instance replication means; +mean = the richer pool is better).

Contrast	Scope	n	Mean (%)	p	Verdict
generic → critical (add g1, g2)	all	40	+0.114	$< 10^{-4}$	significant

Contrast	Scope	n	Mean (%)	p	Verdict
critical → full (add g3, g4)	all	42	+0.047	0.0001	significant
critical → full	TW cells	28	+0.045	0.0003	significant
generic → full (all guided)	all	43	+0.161	$< 10^{-4}$	significant

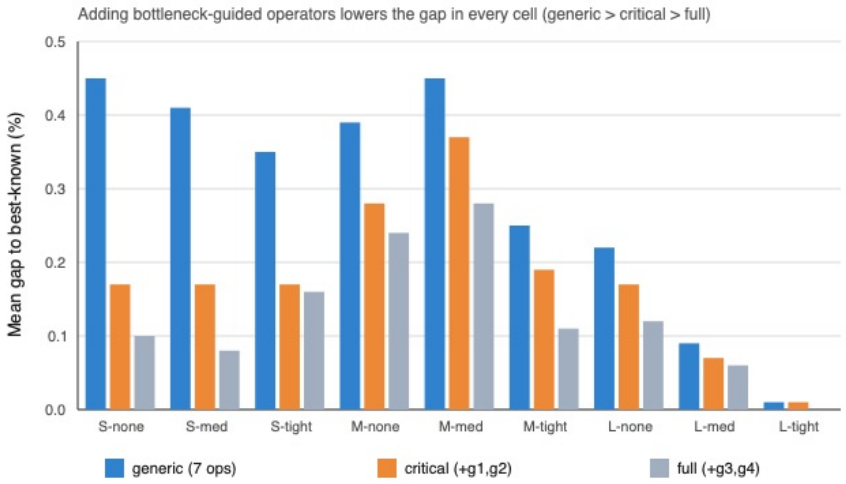

Figure 1: Operator-pool ablation. Adding the bottleneck-guided operators lowers the mean gap to best-known in every one of the nine cells; the ordering generic > critical > full is monotone throughout.

The guided operators lower the objective consistently and significantly, and the effect never flips sign. (On no-time-window cells the g3/g4 gain is partly a selection-weight effect, since they fall back to g1/g2 when no truck is late; the unambiguous new-capability results are generic → critical everywhere and critical → full on the time-window cells.)

Engine components. Holding the pool at *full* and the policy at uniform, we remove one engine component at a time and measure the resulting degradation (Table 4). Best-improvement descent is the dominant contributor and kick restarts are second, both significant everywhere. The Vogel-approximation initial solution has no statistically detectable pooled effect — starting from a random feasible solution reaches essentially the same objective — consistent with a separate sensitivity test in which a purely random start (26–31% worse than VAA) changes the final objective by at most ±0.33%. Removing stochastic

acceptance (making it greedy) does *not* worsen the objective on average, and is marginally better at M and L; the deterministic descent, restarts, and guided operators already reach the attractor without it.

Table 4. Engine-component ablation (leave-one-out; degradation = relative objective increase when removed; Wilcoxon on paired instance replication means).

Component removed	n	Degradation (%)	p	Verdict
best-improvement descent	45	+0.156	$< 10^{-4}$	significant
kick restart	43	+0.069	$< 10^{-4}$	significant
VAA initial solution	45	+0.022	0.114	no significant difference
stochastic acceptance	45	−0.026	0.0014	greedy slightly better

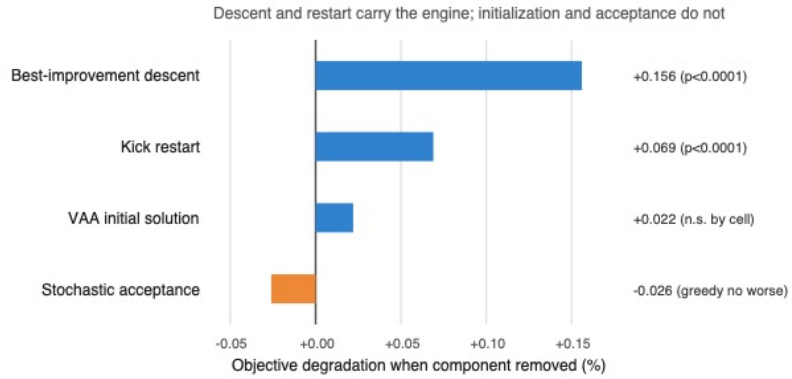

Figure 2: Engine-component leave-one-out ablation. Best-improvement descent and kick restarts are the significant contributors; the VAA initial solution has no statistically significant pooled effect, and removing stochastic acceptance slightly improves the mean objective in this experiment.

Learned operator selection. Finally we vary the selection policy itself, holding pool and components at *full*: uniform random, the base model's tabular Q-learning, and a transfer-trained deep Q-network (Table 5). No learned policy beats uniform by a practically meaningful margin. Across budgets (mean gap to best-known) uniform runs 0.47 → 0.16%, tabular 0.56 → 0.09%, and DQN 0.56 → 0.26% from 50 to 3,000 iterations: at 50 iterations uniform is *better*

than tabular (+0.08%, $p = 0.0055$), at 3,000 tabular edges uniform (+0.07%, $p < 10^{-4}$), and the transfer DQN never wins at any budget and is significantly worse at 1,000 and 3,000. All selection effects are ≤ 0.17 percentage points — below the guided-operator effect of Table 3 and an order of magnitude below the method-level differences of Table 2 — and their direction flips with budget.

Crucially, the same picture holds on the no-time-window cells alone, which are exactly the original problem of Shahmardan and Sajadieh (2020) with all trucks available at time zero: at 1,000 iterations tabular Q-learning is indistinguishable from uniform (mean -0.05%, $p = 0.52$, $n = 58$) and the transfer DQN is significantly worse (uniform better by $+0.14\%$, $p = 0.0012$; tabular better by $+0.09\%$, $p < 10^{-3}$). The negative result is therefore not an artifact of the time-window extension; it already holds in the base setting where the original model introduces learned selection.

Table 5. Selection-policy effects at 1,000 iterations (two-sided Wilcoxon on paired instance replication means; positive mean = first method better).

Comparison	Mean diff (%)	p	Verdict
tabular vs uniform	-0.01	0.150	no difference
tabular vs DQN	$+0.10$	$< 10^{-4}$	DQN worse
uniform vs DQN	$+0.11$	$< 10^{-4}$	DQN worse

Across the three experiments, descent has the largest measured effect. Guided moves and restarts provide smaller but repeatable gains. The initial solution, stochastic acceptance, and learned selection have little practical effect once those parts are present. One tuning instance illustrates the pattern: several selector variants converged to exactly the CP-SAT incumbent obtained after 240 seconds. This is only a point check, but it shows how the local search can erase differences created earlier in a run.

6.3 Tardiness-weight sensitivity

The objective (16) weights total tardiness by λ; our experiments fix $\lambda = 1$. To justify this modelling choice we trace the makespan–tardiness trade-off by sweeping $\lambda \in \{0, 0.25, 0.5, 1, 2, 4, 8\}$ on the *tuning* pool (all six time-window cells, five instances $\times$ three replications, 1,000 iterations, seeds shared across λ so that only the objective weight differs). For each run we record the *physical* makespan and total tardiness of the returned schedule; to combine cells of very different magnitude we min–max normalize each metric across the λ grid per instance and average (Figure 4).

The absolute trade-off is mild. Across the sweep, mean makespan rises by only about 1% (e.g., S 2060.8 $\rightarrow$ 2084.1), while mean tardiness falls by 2–3% (e.g., L 87215 $\rightarrow$ 86306). At $\lambda = 1$, tardiness is already near its minimum (normalized 0.14, versus 0.04 at the extreme $\lambda = 8$) at only a moderate makespan cost

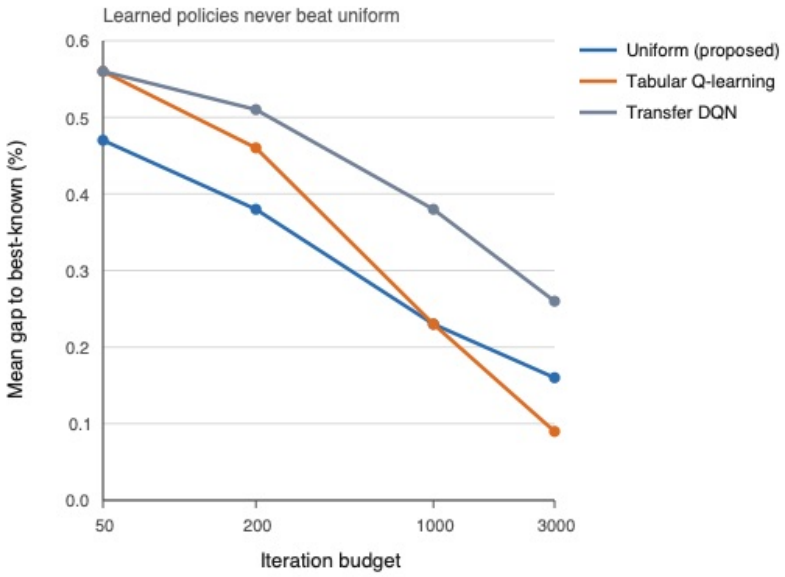

Figure 3: Selection-policy budget sweep. Across budgets from 50 to 3,000 iterations, neither learned policy beats uniform random selection by a practically meaningful margin; the transfer DQN is consistently worse and the tabular policy only edges uniform at the largest budget.

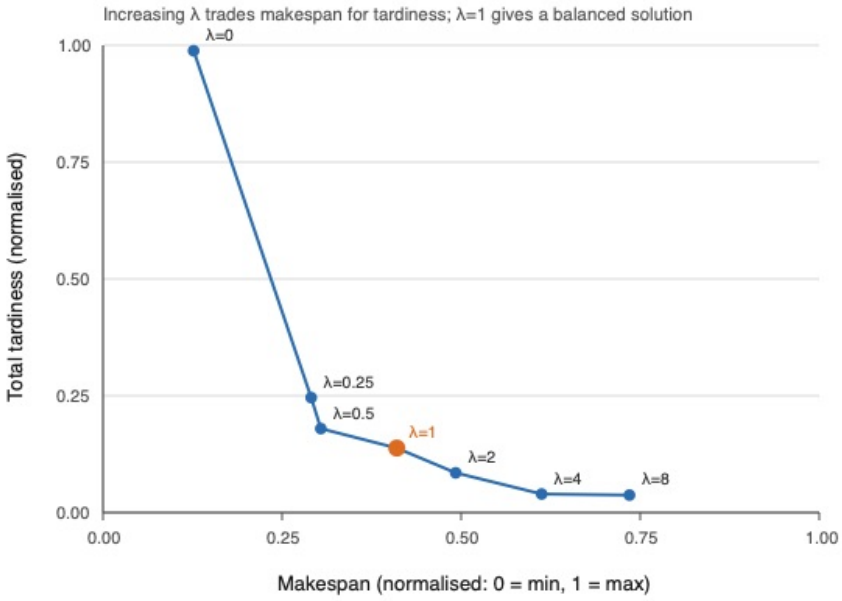

Figure 4: Tardiness-weight sensitivity. As λ grows, the search trades a higher makespan for lower tardiness along a convex frontier (normalized makespan 0.13 to 0.74, normalized tardiness 0.99 to 0.04). Most of the achievable tardiness reduction is bought by the first increment away from $\lambda = 0$, after which returns diminish while the makespan penalty keeps rising.

(normalized 0.41). We use $\lambda = 1$ because it also has a direct interpretation: one unit of makespan time is exchanged for one unit of tardiness time. The sweep is reported as design justification, not as a tuned result, and uses only the tuning pool.

7. Discussion

The selector result should be interpreted within the search process used here. A candidate can be evaluated in tens of microseconds, so an unhelpful operator choice is inexpensive. More importantly, every new incumbent is followed by a deterministic descent. Two runs that enter the same basin can therefore end at the same schedule even if their preceding moves differ. The budget curves suggest that this happens before 1,000 iterations for most test instances.

There is also little information for a policy to predict. Instances are static, and all releases are known before optimization begins. This is quite different from online dock control, where arrivals are revealed over time, or from a simulation model in which evaluating one move is costly. Learned selection may matter more in either setting. It may also matter on instances large enough that descent cannot reach saturation within the available budget. We did not test those cases, so they remain hypotheses rather than consequences of the present experiments.

What the results do support is a narrower recommendation. A learned selector should be compared with uniform sampling inside the same engine and under the same evaluation budget. Without that control, gains due to descent, restart, or a stronger move set can be attributed to the policy itself.

8. Conclusion

This study adds release times and soft due dates to compound-truck scheduling with partial unloading. The MILP and CP-SAT formulations also provide a check on the heuristic results. On the exact-referenced subsets, VAA-GILS finished within a fraction of a percent of the CP-SAT incumbent; optimality was proven only for the small no-window cases. For the larger windowed instances, it found the best schedules among the methods tested in a few seconds, while CP-SAT did not return a feasible schedule within its limit.

The ablations change how that result should be understood. Most of the gain is due to best-improvement descent, with additional gains from bottleneck moves and restarts. In this engine, tabular Q-learning and the transfer DQN did not improve meaningfully on uniform operator sampling. Nor did the final result depend much on the starting solution or simulated-annealing acceptance. These findings are specific to the present benchmark. An online version with unrevealed arrivals would provide a more demanding test of adaptive policies; tighter bounds for the larger time-window cases are another priority.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude Code (Anthropic) in order to assist with software for the computational experiments, generation of result figures, and drafting and language editing of the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

Assadi, M. T., & Bagheri, M. (2016). Differential evolution and population-based simulated annealing for truck scheduling problem in multiple door cross-docking systems. *Computers & Industrial Engineering, 96*, 149–161. https://doi.org/10.1016/j.cie.2016.03.021

Bodnar, P., de Koster, R. B. M., & Azadeh, K. (2017). Scheduling trucks in a cross-dock with mixed service mode dock doors. *Transportation Science, 51*(1), 112–131. https://doi.org/10.1287/trsc.2015.0612

Boysen, N., & Fliedner, M. (2010). Cross dock scheduling: Classification, literature review and research agenda. *Omega, 38*(6), 413–422.

Joo, C. M., & Kim, B. S. (2013). Scheduling compound trucks in multi-door cross-docking terminals. *International Journal of Advanced Manufacturing Technology, 64*, 977–988.

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science, 220*(4598), 671–680.

Konur, D., & Golias, M. M. (2013a). Analysis of different approaches to cross-dock truck scheduling with truck arrival time uncertainty. *Computers & Industrial Engineering, 65*(4), 663–672. https://doi.org/10.1016/j.cie.2013.05.009

Konur, D., & Golias, M. M. (2013b). Cost-stable truck scheduling at a cross-dock facility with unknown truck arrivals: A meta-heuristic approach. *Transportation Research Part E: Logistics and Transportation Review, 49*(1), 71–91. https://doi.org/10.1016/j.tre.2012.06.007

Kool, W., van Hoof, H., & Welling, M. (2019). Attention, learn to solve routing problems! In *Proceedings of the International Conference on Learning Representations*.

Korukoğlu, S., & Ballı, S. (2011). An improved Vogel's approximation method for the transportation problem. *Mathematical and Computational Applications, 16*(2), 370–381.

Ladier, A.-L., & Alpan, G. (2016a). Cross-docking operations: Current research versus industry practice. *Omega, 62*, 145–162.

Ladier, A.-L., & Alpan, G. (2016b). Robust cross-dock scheduling with time windows. *Computers & Industrial Engineering, 99*, 16–28. https://doi.org/10.1016/j.cie.2016.07.003

Larbi, R., Alpan, G., Baptiste, P., & Penz, B. (2011). Scheduling cross docking operations under full, partial and no information on inbound arrivals. *Computers & Operations Research, 38*(6), 889–900. https://doi.org/10.1016/j.cor.2010.10.003

Li, Y., Mohammadi, M., Zhang, X., Lan, Y., & van Jaarsveld, W. (2026). Integrated trucks assignment and scheduling problem with mixed service mode docks: A Q-learning based adaptive large neighborhood search algorithm. *European Journal of Operational Research, 333*(1), 117–137. https://doi.org/10.1016/j.ejor.2025.12.036

Lourenço, H. R., Martin, O. C., & Stützle, T. (2019). Iterated local search: Framework and applications. In *Handbook of metaheuristics* (3rd ed.). Springer.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature, 518*, 529–533.

Molavi, D., Shahmardan, A., & Sajadieh, M. S. (2018). Truck scheduling in a cross docking systems with fixed due dates and shipment sorting. *Computers & Industrial Engineering, 117*, 29–40. https://doi.org/10.1016/j.cie.2018.01.009

Perron, L., & Didier, F. (2024). *Google OR-Tools CP-SAT solver* (Version 9.x) [Computer software]. https://developers.google.com/optimization

Rijal, A., Bijvank, M., & de Koster, R. (2019). Integrated scheduling and assignment of trucks at unit-load cross-dock terminals with mixed service mode dock doors. *European Journal of Operational Research, 278*(3), 752–771. https://doi.org/10.1016/j.ejor.2019.04.028

Shahmardan, A., & Sajadieh, M. S. (2020). Truck scheduling in a multi-door cross-docking center with partial unloading — Reinforcement learning-based simulated annealing approaches. *Computers & Industrial Engineering, 139*, Article 106134.

Van Belle, J., Valckenaers, P., & Cattrysse, D. (2012). Cross-docking: State of the art. *Omega, 40*(6), 827–846.

Van Belle, J., Valckenaers, P., Vanden Berghe, G., & Cattrysse, D. (2013). A tabu search approach to the truck scheduling problem with multiple docks and time windows. *Computers & Industrial Engineering, 66*(4), 818–826. https://doi.org/10.1016/j.cie.2013.09.024

Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning, 8*, 279–292.

Xi, X., Liu, C., Wang, Y., & Lee, L. H. (2020). Two-stage conflict robust optimization models for cross-dock truck scheduling problem under uncertainty. *Transportation Research Part E: Logistics and Transportation Review, 144*, Article 102123. https://doi.org/10.1016/j.tre.2020.102123

Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P. S., & Chi, X. (2020). Learning to dispatch for job shop scheduling via deep reinforcement learning. In *Advances in Neural Information Processing Systems* (Vol. 33).